

Documentación Técnica: Evaluá tu Compra

1. Descripción General

Este proyecto es un componente web de "Micro-frontend" diseñado para capturar la satisfacción del cliente y comentarios post-compra. Se caracteriza por ser un archivo único (single-file component) que integra estructura (HTML), estilo (CSS) y lógica (JavaScript).

El objetivo principal es proporcionar una interfaz reactiva donde el ambiente visual cambia según la calificación del usuario, e incluye un sistema de moderación automática de comentarios.

2. Tecnologías y Herramientas Utilizadas

- **HTML5:** Estructura semántica del contenido.
- **CSS3:** Diseño visual, variables CSS (Custom Properties), animaciones y diseño responsivo.
- **JavaScript (ES6+):** Lógica del cliente, manipulación del DOM y manejo de eventos.
- **Google Fonts / System Fonts:** Uso de la pila de fuentes del sistema (Inter, San Francisco, Segoe UI) para optimizar el rendimiento sin peticiones externas.

3. Arquitectura del Código

El código está contenido en un solo archivo .html para facilitar su portabilidad. A continuación se detallan las tres capas:

3.1. Capa de Estilos (CSS)

El diseño utiliza una arquitectura basada en **Variables CSS** definida en :root. Esto facilita el mantenimiento y el cambio de temas visuales.

Variables Clave:

```
:root {
  --primary-color: #6366f1; /* Color principal (Botones, slider) */
  --card-bg: #ffffff;      /* Fondo de la tarjeta */
  --shadow-soft: ...;      /* Sistema de sombras */
}
```

Aspectos destacados de UI/UX:

- **Transiciones:** El body tiene una transición de 0.8s para suavizar el cambio de color de fondo.
- **Animaciones:** Se usan @keyframes slideUp y fadeIn para dar entrada a los mensajes de respuesta.
- **Slider Personalizado:** Se sobrescriben los estilos nativos de input[type="range"] para crear una experiencia visual consistente en diferentes navegadores (webkit).

3.2. Capa de Lógica (JavaScript)

La lógica se centra en dos funcionalidades principales: la reacción visual (cambio de fondo) y el procesamiento de texto (filtro).

A. Sistema de Calificación (Slider)

El slider tiene un rango de 0 a 2:

- 0: Experiencia Buena (Verde)
- 1: Experiencia Regular (Amarillo)
- 2: Experiencia Mala (Rojo)

Función `updateBackground(value)`:

Recibe el valor del input y mapea el color correspondiente desde un objeto colors. Si el valor no existe, hace fallback al color por defecto.

B. Sistema de Filtrado de Comentarios

Para mantener un ambiente seguro, se implementa un filtro de palabras prohibidas del lado del cliente.

Función `filtrarTexto(texto)`:

1. Divide el texto en un array de palabras.
2. Itera sobre cada palabra limpiando signos de puntuación (.,,:!/?¿¡).
3. Compara la palabra limpia con el array palabrasProhibidas.
4. Si hay coincidencia, reemplaza la palabra con asteriscos (*) manteniendo la longitud original.
5. Reconstruye la frase.

Array de configuración:

```
const palabrasProhibidas = [
  'queso', 'huevo', 'leche', 'feo', 'horrible'
];
```

4. Guía de Mantenimiento y Extensión

Cómo cambiar la paleta de colores

Para modificar los colores de la interfaz (botones, textos), edita las variables dentro de la etiqueta `<style>` en la sección `:root`.

Para modificar los colores de fondo reactivos (semáforo), edita el objeto `colors` dentro de la función `updateBackground` en el `<script>`:

```
const colors = {  
  '0': '#NUEVO_COLOR_VERDE',  
  '1': '#NUEVO_COLOR_AMARILLO',  
  '2': '#NUEVO_COLOR_ROJO'  
};
```

Cómo agregar nuevas palabras prohibidas

Simplemente añade las palabras (en minúsculas) al array `palabrasProhibidas` dentro del bloque `<script>`:

```
const palabrasProhibidas = [  
  'queso',  
  'huevo',  
  'leche',  
  'NUEVA_PALABRA_1',  
  'NUEVA_PALABRA_2'  
];
```

Integración con Backend (Futuro)

Actualmente, el botón "Enviar" solo simula el envío. Para conectar con un servidor real, se debe modificar el `addEventListener` del botón `sendBtn`:

1. Eliminar la lógica de visualización inmediata.
2. Implementar `fetch()` o `XMLHttpRequest`.
3. Enviar `ratingSlider.value` y `commentInput.value` al endpoint deseado.

5. Estructura de Archivos (Sugerida para escalar)

Si el proyecto crece, se recomienda separar el código en tres archivos:

1. `index.html`: Solo la estructura.
2. `styles.css`: Todo el contenido de `<style>`.
3. `app.js`: Todo el contenido de `<script>`.